

From Neural Network to Hardware Accelerator

Understanding the ONNX to RISC-V + Gemmini Compilation Pipeline

A Practical Guide for Systems Programmers

Technical Documentation

July 10, 2026

Plain English Summary

What This Pipeline Actually Does

This document describes how we take a neural network model (specifically ResNet-18) and turn it into a program that runs on a RISC-V processor with a Gemmini hardware accelerator. In simple terms:

- **Input:** A ResNet-18 model saved as an ONNX file (about 46 MB of network structure and weights)
- **Output:** A static executable file (about 50 MB) that runs on RISC-V Linux
- **What happens:** The compiler translates the neural network operations into RISC-V CPU code and accelerator library calls
- **How it works:** The model weights become constant data in the executable; the model operations become function calls to a runtime library that talks to the accelerator

Why We Need This

- Neural networks are computationally intensive (millions of multiply-add operations)
- CPUs alone are too slow for practical inference
- Accelerators like Gemmini can perform matrix operations 10-100x faster
- We need a way to compile the model to use the accelerator automatically
- This pipeline does that: it converts high-level operations into accelerator instructions

CPU Programmer's Mental Model

If you understand CPU programming, you already know most of what's happening here. Here's a quick mapping:

Compiler/Accelerator Term	CPU Programming Analogy
ONNX Model	A graph of function calls and data dependencies
Tensor	A multi-dimensional array with metadata (shape, rank, strides)
Weights	Global <code>const</code> arrays initialized with pre-trained values
Lowering	Translating high-level code to lower-level code (like Python $\rightarrow$ C $\rightarrow$ asm)
MLIR Dialect	Different intermediate languages within the compiler toolchain
Pass	A transformation that modifies the program representation
Memref	A C struct: <code>{void* data; int64_t* shape; int rank;}</code>
OMTensor	A standardized C struct for passing tensor data between functions
Krnl.call	A function call to a runtime library
Runtime Library	Like libc, but for tensor operations
ABI	Calling convention and binary interface rules
Static Linking	Copying library code directly into your executable (gcc <code>-static</code>)
RoCC	A coprocessor interface, similar to SIMD instructions
Gemmini	A systolic array accelerator (SIMD on steroids)
Scratchpad	On-chip memory managed explicitly (like L2 cache you control)

Table 1: Quick translation guide for CPU programmers.

From CPU to Accelerator

Gemmini is like... a 16x16 SIMD unit that can do 256 multiplications per cycle, with its own local memory. It's connected to the CPU through custom instructions (RoCC).

RoCC (Rocket Custom Coprocessor) is like... a coprocessor interface similar to how x86 has SIMD instructions, but more flexible. The CPU issues a special instruction, and the accelerator executes it.

Systolic array is like... a 2D pipeline where data flows in a wave, like a conveyor belt of calculations. Each processing element does multiply-add and passes results to its neighbors.

Weight-stationary dataflow is like... keeping the filter weights in the accelerator's local memory and streaming the input data through. Similar to keeping a loop invariant in a register.

Scratchpad memory is like... L2 cache that you manage explicitly. You decide what goes there and when to load/store data.

im2col is like... converting a convolution into a matrix multiply. It's like unrolling a nested loop to use optimized matrix multiply routines.

The Big Picture in One Diagram

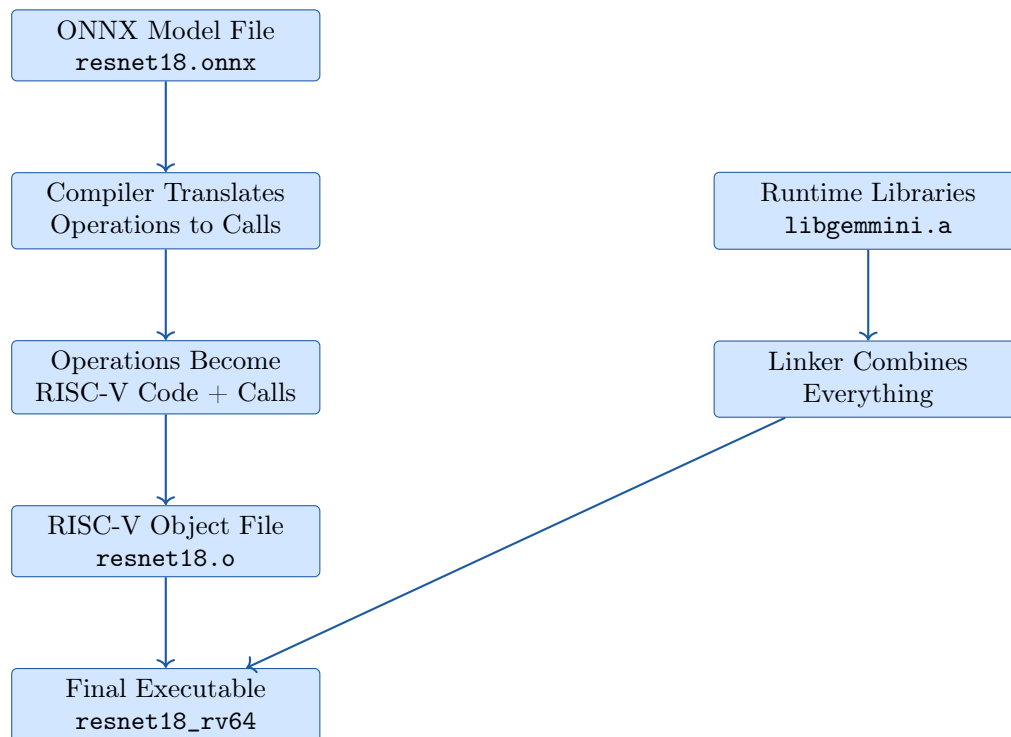


Figure 1: High-level pipeline flow. The compiler transforms the model into RISC-V code, which is then linked with runtime libraries to produce the final executable.

Key Terms

ONNX-MLIR: The compiler that reads ONNX models and generates MLIR **MLIR:** Multi-Level Intermediate Representation - a compiler framework with multiple languages **LLVM IR:** Low-level virtual machine intermediate representation (like assembly with unlimited registers) **RISC-V:** Open instruction set architecture for CPUs **Gemmini:** Hardware accelerator for matrix operations **RoCC:** Rocket Custom Coprocessor - interface for accelerators **ResNet-18:** A neural network architecture with 18 layers **Static Linking:** Combining all code into one executable (no shared libraries at runtime) **OMTensor:** Runtime data structure for passing tensors between compiler-generated code and runtime libraries

Contents

Plain English Summary	1
Key Terms	3
1 Introduction: What We're Building	6
1.1 What Problem Does This Solve?	6
1.2 Why Two Compilers?	7
1.3 What's in This Document	7

2	The Runtime Directory: Understanding the Bridge	8
2.1	What Each Runtime File Does	9
2.2	The Runtime Flow in Practice	10
2.3	Understanding the Gemmini ABI Headers	10
3	Pipeline Overview: The Nine Stages	10
4	Stage-by-Stage Breakdown	11
4.1	Stage 1: ONNX to ONNX Dialect	11
4.1.1	What Problem Does This Solve?	11
4.1.2	What It Does	12
4.1.3	What It Looks Like	12
4.2	Stage 2: ONNX to Krnl/Gemmini MLIR	13
4.2.1	What Problem Does This Solve?	13
4.2.2	What It Does	13
4.2.3	Support Check Example	13
4.2.4	What It Looks Like	14
4.2.5	What Operations Get Accelerated?	14
4.3	Stage 3: MLIR to LLVM Dialect	14
4.3.1	What Problem Does This Solve?	14
4.3.2	What It Does	15
4.3.3	What It Looks Like	15
4.4	Stage 4: LLVM Dialect to LLVM IR	15
4.4.1	What Problem Does This Solve?	15
4.4.2	What It Does	16
4.4.3	What It Looks Like	16
4.5	Stage 5: LLVM IR to RISC-V Object File	16
4.5.1	What Problem Does This Solve?	16
4.5.2	What It Does	17
4.5.3	What It Looks Like	17
4.6	Stage 6: ONNX-MLIR C Runtime Library	17
4.6.1	What Problem Does This Solve?	17
4.6.2	What It Does	18
4.6.3	What the Runtime Provides	18
4.7	Stage 7: Gemmini Runtime Library	18
4.7.1	What Problem Does This Solve?	18
4.7.2	What It Does	18
4.7.3	What's Inside Each Runtime Function	19
4.7.4	Implementation Example (Conceptual)	19
4.8	Stage 8: Runner Program	19
4.8.1	What Problem Does This Solve?	19
4.8.2	What It Does	20
4.8.3	What It Looks Like	20
4.9	Stage 9: Static Linking	20
4.9.1	What Problem Does This Solve?	20
4.9.2	What It Does	21
4.9.3	Link Order Matters!	21
4.9.4	What the Final Executable Contains	21

5	Deep Trace: One Convolution and All Runtime Symbols	22
5.1	Stage 1: ONNX File to ONNX Dialect	22
5.2	Stage 2: ONNX Dialect to Gemmini/Krnl MLIR	22
5.3	Stage 3: Krnl to LLVM Dialect	23
5.4	Stage 4: LLVM Dialect to LLVM IR	23
5.5	Stage 5: LLVM IR to RISC-V Object	24
5.6	Stages 6-7: Runtime Libraries	24
5.7	Stage 8: Runner	24
5.8	Stage 9: Static Linking	24
5.9	The Convolution Journey Complete	25
5.10	Trace Matrix for All Gemmini Runtime Functions	25
6	Verification and Debugging	30
6.1	Checking the Final Executable	30
6.2	Common Issues and Solutions	31
6.3	Common Misunderstandings	31
7	Complete Build Script	31
8	Quick Reference: What Each Command Does	33
9	Summary: What We've Built	34

1 Introduction: What We're Building

This document describes the complete process of taking a ResNet-18 neural network model (saved as an ONNX file) and turning it into a RISC-V executable that uses the Gemmini hardware accelerator. The final output is a self-contained static executable that can run on RISC-V hardware with Gemmini support.

What's in the Final Executable

The executable `resnet18_rv64` (about 50 MB) contains:

- The compiled ResNet-18 model code (which layer to run, in what order)
- All model weights (converted to constant data in `.rodata`)
- The Gemmini runtime library (code that talks to the accelerator)
- The ONNX-MLIR runtime library (tensor management)
- System libraries (math functions, memory management)
- A test harness (the `main()` function)

1.1 What Problem Does This Solve?

- **Neural networks are slow on CPUs:** Each inference involves millions of multiply-add operations. A typical CNN layer might need 100 million operations for a single image.
- **Hardware accelerators exist but need special code:** Gemmini is a systolic array that can do many multiplications in parallel, but you need to tell it what to do through custom instructions.
- **We need automation:** Manually rewriting neural networks to use accelerators is impossible for large models. We need a compiler to do it automatically.
- **This pipeline provides the automation:** It takes the high-level model description and generates executable code that uses the accelerator efficiently.

Think of it like compiling C

When you compile C code:

1. You write high-level code (`for` loops, function calls)
2. The compiler translates it to assembly instructions
3. The assembler creates object files
4. The linker combines them into an executable

This pipeline does the same thing, but for neural networks:

1. The model is described as operations (convolution, matrix multiply, activation)

2. The compiler translates these to function calls (to the accelerator runtime)
3. The runtime library implements those functions using hardware instructions
4. Everything is linked into one executable

1.2 Why Two Compilers?

You might wonder: why can't we just use one compiler? The answer is division of labor:

Compiler	What It's Good At	What It Produces
ONNX-MLIR (LLVM-based)	Understanding neural network structure, tensor shapes, and operations	RISC-V object code for the model itself
GCC (GNU Compiler Collection)	Compiling C/C++ runtime libraries, system code	Runtime libraries (OMTensor, Gemmini runtime)

The two compilers meet in the middle through the ABI (Application Binary Interface):
 - ONNX-MLIR generates code that calls functions like `om_gemmini_conv_f32()` - GCC compiles the implementation of those functions - The linker connects the calls to the implementations

Think of it like this

Imagine you're writing a program that uses the `sin()` function:

1. You write code that calls `sin(x)`
2. Your compiler generates a call to an external symbol
3. The linker connects that call to the math library (`-lm`)
4. The math library is pre-compiled and contains the actual implementation

The same thing happens here:

1. The compiled model calls `om_gemmini_conv_f32(...)`
2. The linker connects that call to the Gemmini runtime library
3. The Gemmini runtime library contains the actual accelerator implementation

1.3 What's in This Document

This document walks through the entire pipeline step by step. For each stage, you'll learn:

- **What problem it solves** - why we can't skip this stage
- **What it does in plain English** - the core transformation
- **CPU programming analogy** - how this is like something you already know

- **Before and after examples** - what the code looks like at each stage
- **What goes wrong if we skip it** - the consequences of omission

2 The Runtime Directory: Understanding the Bridge

Before diving into the pipeline, let's look at the runtime directory. This is where the bridge between the compiled model and the accelerator lives.

What's in the Runtime Directory

The runtime directory (`src/Accelerators/Gemmini/Runtime/`) contains:

- **OMRuntimeGemmini.hpp/cpp**: The main implementation of accelerator operations
- **gemmini-hardware-abi/**: Bundled Gemmini hardware headers
- **CMakeLists.txt**: Build configuration
- **FILES_EXPLAINED.md**: Documentation

This directory is the translator between compiler-generated calls and actual accelerator instructions.

Understanding the Runtime Files

Let's map these to CPU concepts:

- `OMRuntimeGemmini.hpp` is like a header file that declares functions you'll call
- `OMRuntimeGemmini.cpp` is like the implementation of those functions
- `gemmini-hardware-abi/` is like the CPU's instruction set manual (but for the accelerator)
- The compiled `libgemmini_f32_rv64.a` is like a static library you link with

2.1 What Each Runtime File Does

File	CPU Programmer's Understanding	What It Actually Does
<code>OMRuntimeGemmini.hpp</code>	Header file declaring accelerator functions	Declares <code>om_gemmini_conv_f32</code> , <code>om_gemmini_matmul</code> , etc. with <code>extern "C"</code>
<code>OMRuntimeGemmini.cpp</code>	Implementation of those functions	Implements convolution, matrix multiply, pooling, activation functions
<code>gemmini-hardware-abi/</code>	CPU instruction set documentation	Defines the low-level interface to Gemmini (RoCC instructions, register layout)
<code>CMakeLists.txt</code>	Makefile alternative	Controls compilation, finds headers, sets flags

Think of it like hardware drivers

The Gemmini runtime is like a device driver for a GPU or a custom accelerator:

- `OMRuntimeGemmini.cpp` is the driver implementation
- It translates high-level operations (like "do a convolution") into low-level hardware commands
- `gemmini-hardware-abi/` is the hardware specification
- The driver knows how to configure the hardware, move data, and execute operations

When your compiled model calls `om_gemmini_conv_f32()`, it's like calling `write()` on a device file. The runtime driver translates it into the actual hardware commands.

2.2 The Runtime Flow in Practice

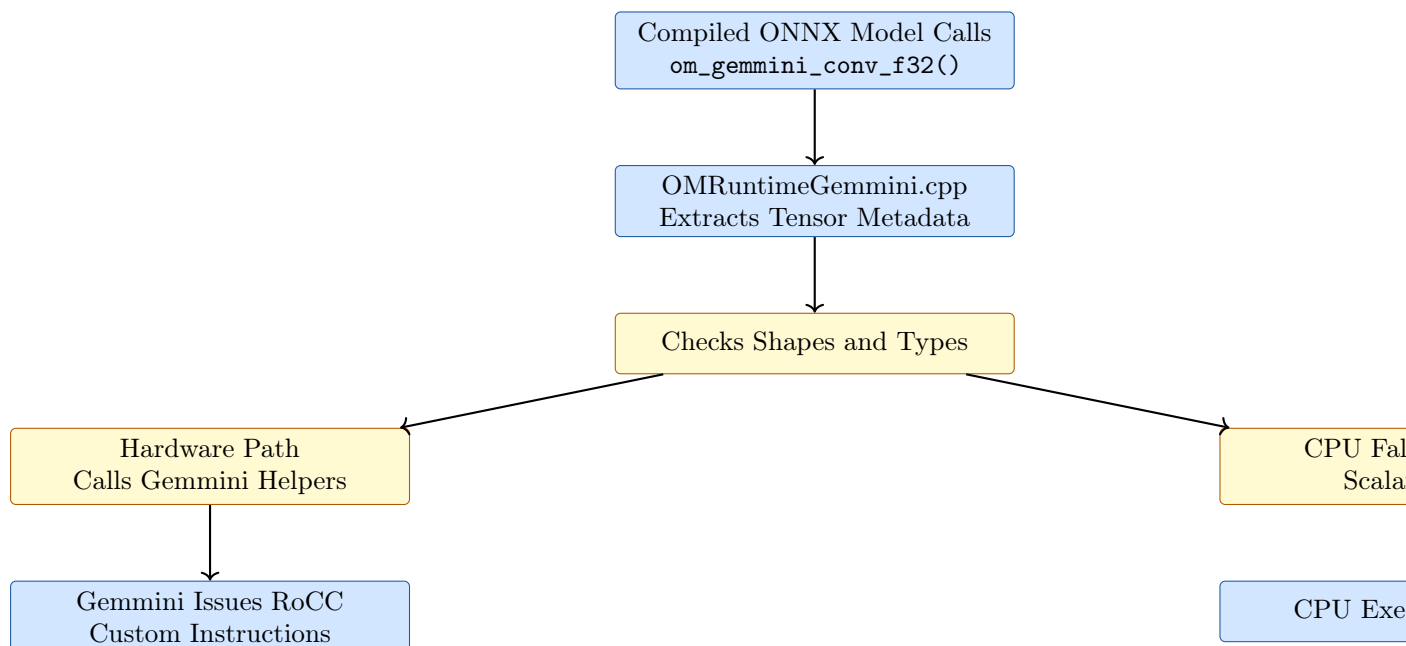


Figure 2: Runtime execution flow. The runtime decides whether to use the accelerator or fall back to CPU.

2.3 Understanding the Gemmini ABI Headers

The `gemmini-hardware-abi/` directory bundles the headers needed to talk to Gemmini. Here's what each file does:

Header File	CPU Programmer's Understanding	What It Defines
<code>gemmini.h</code>	Main API header	<code>tilled_matmul_auto</code> , <code>tilled_conv_auto</code> , configuration functions
<code>gemmini_params.h</code>	CPU feature flags	Array size (16), data types (<code>int8_t</code> , <code>int32_t</code>), scratch-pad sizes
<code>gemmini_nn.h</code>	Neural network helper functions	High-level conv/pooling helpers, parameter structs
<code>gemmini_counter.h</code>	Performance counters	Cycle counters, DMA counters, etc.
<code>xcustom.h</code>	Custom instruction macros	Macros that emit actual RISC-V custom instructions

3 Pipeline Overview: The Nine Stages

Here's the complete pipeline at a glance. The diagram below shows what each stage does and what files it produces.

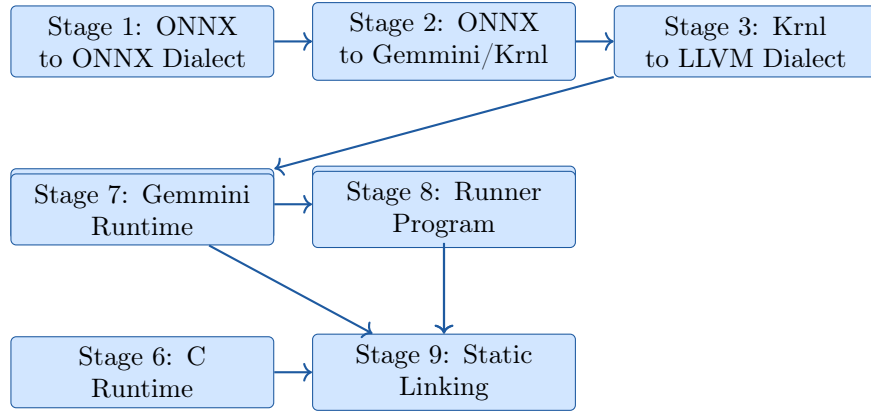


Figure 3: The nine stages of the compilation pipeline. Stages 1-5 produce the model object; Stages 6-8 produce support libraries; Stage 9 links everything together.

Stage	Input	Output	What Happens
1	ONNX model	ONNX dialect MLIR	Parse the model into compiler's internal representation
2	ONNX dialect	Krnl/Gemmini MLIR	Replace operations with runtime calls where possible
3	Krnl/Gemmini	LLVM dialect MLIR	Convert to LLVM's internal representation
4	LLVM dialect	LLVM IR text	Export as standard LLVM assembly
5	LLVM IR	RISC-V object file	Generate RISC-V machine code
6	C runtime sources	Static archive	Build ONNX-MLIR tensor management library
7	C++ runtime sources	Static archive	Build Gemmini accelerator library
8	Runner C source	Object file	Build test harness with <code>main()</code>
9	Objects + archives	Static executable	Link everything into one ELF file

4 Stage-by-Stage Breakdown

4.1 Stage 1: ONNX to ONNX Dialect

Think of it like reading a header file

Stage 1 is like the compiler reading a `.h` file. It parses the ONNX file and builds an internal representation. The compiler doesn't generate code yet - it just understands what the model looks like.

4.1.1 What Problem Does This Solve?

The ONNX file is a binary protobuf format. We need to convert it into a format that the compiler can manipulate. This stage parses the binary and creates an MLIR representation.

4.1.2 What It Does

- Reads the ONNX protobuf file
- Parses the graph structure (which operations, in what order)
- Imports tensor initializers (weights) as constants
- Creates MLIR operations like `onnx.Conv`, `onnx.MatMul`
- Emits `resnet18_onnxir.onnx.mlir`

4.1.3 What It Looks Like

Before (ONNX file):

```
1 resnet18.onnx # binary protobuf file, about 46 MB
```

After (ONNX dialect MLIR):

```
1 %0 = "onnx.Conv"(%input, %weights, %bias) {
2   dilations = [1, 1],
3   group = 1,
4   kernel_shape = [7, 7],
5   pads = [3, 3, 3, 3],
6   strides = [2, 2]
7 } : (tensor<1x3x224x224xf32>,
8     tensor<64x3x7x7xf32>,
9     tensor<64xf32>) -> tensor<1x64x112x112xf32>
```

Listing 1: Stage 1 Output Example

Understanding the syntax

This is like a C function call:

- `%0` is the result variable (like `int result =`)
- `"onnx.Conv"` is the operation name (like a function name)
- `(%input, %weights, %bias)` are the arguments
- The attributes (`strides`, `pads`) are like function parameters
- The types are like `tensor<...>` describing the data shapes

Checkpoint: What to Verify

After Stage 1, verify:

- The file exists: `resnet18_onnxir.onnx.mlir`
- Search for `"onnx.Conv"` to find convolution operations
- Search for `"onnx.MatMul"` to find matrix multiplications
- Check that weights appear as constants

If any of these are missing, the ONNX file may be corrupted or unsupported.

4.2 Stage 2: ONNX to Krnl/Gemmini MLIR

Think of it like inlining functions

This stage is like the compiler deciding to replace some function calls with inline code or with calls to specialized libraries. For operations that the accelerator can handle, it replaces the high-level operation with a call to the accelerator runtime.

4.2.1 What Problem Does This Solve?

The ONNX dialect is high-level. We need to decide which operations will run on the accelerator and which will run on the CPU. This stage makes those decisions and rewrites the operations accordingly.

4.2.2 What It Does

- Examines each operation to see if Gemmini can accelerate it
- For supported operations (convolution, matmul, pooling, etc.):
 - Checks shapes, types, and attributes
 - Replaces the operation with a runtime function call
- For unsupported operations:
 - Lowers them to Krnl dialect (loops, memory operations)
- Weight constants become `krnl.global` symbols
- Emits `resnet18_krnl.onnx.mlir`

4.2.3 Support Check Example

For a convolution to be accelerated, all of these must be true:

- Rank-4 tensor (batch, channels, height, width)
- Float32 data type
- Group = 1 (no grouped convolution)
- Dilation = 1
- Equal stride in H and W
- Symmetric padding

If any check fails, the convolution stays in ONNX dialect and goes to the CPU path.

4.2.4 What It Looks Like

Before (ONNX dialect):

```
1 %result = "onnx.Conv"(%input, %weights, %bias) { ... }
```

After (Krnl/Gemmini MLIR):

```
1 krnl.call @om_gemmini_conv_f32(
2   %output,          # where to write the result
3   %input,           # input tensor
4   %weights,         # weight tensor
5   %stride,          # stride value (e.g., 2)
6   %padding          # padding value (e.g., 3)
7 )
```

Understanding the transformation

This is like replacing:

```
1 // High-level operation
2 result = convolution(input, weights, bias, stride=2, pad=3)
3
4 // With a function call to a specialized library
5 conv2d_optimized(output, input, weights, bias, 2, 3);
```

The compiler says: "I know how to do convolution, but the accelerator library can do it faster, so I'll call that instead."

4.2.5 What Operations Get Accelerated?

Operation	Count in ResNet-18	What Happens
Conv2D	20	om_gemmini_conv_f32
BatchNorm	20	om_gemmini_batchnorm_f32
ReLU	17	om_gemmini_relu_f32
Add (residual)	8	om_gemmini_add_f32
MaxPool	1	om_gemmini_maxpool_f32
GlobalAvgPool	1	om_gemmini_globalavgpool_f32
GEMM (fully connected)	1	om_gemmini_gemm_f32_bias
Softmax	1	om_gemmini_softmax_f32

4.3 Stage 3: MLIR to LLVM Dialect

Think of it like generating assembly

This stage is like the compiler moving from a language with high-level constructs (loops, arrays) to something closer to assembly (registers, memory addresses, function calls).

4.3.1 What Problem Does This Solve?

MLIR has many dialects. We need to lower to a single, low-level dialect that LLVM can understand. LLVM dialect is the MLIR representation that maps directly to LLVM IR.

4.3.2 What It Does

- Converts Krnl calls to LLVM function calls
- Converts Krnl globals to LLVM global variables
- Converts memrefs to pointer-based operations
- Creates OMTensor* wrappers for crossing the runtime boundary
- Emits resnet18.onnx.mlir (LLVM dialect MLIR)

4.3.3 What It Looks Like

Before (Krnl/Gemmini MLIR):

```
1 krnl.call @om_gemmini_conv_f32(%out, %x, %w, %stride, %pad)
2 : (!krnl.OMTensorType, !krnl.OMTensorType, !krnl.OMTensorType, i64,
   i64) -> ()
```

After (LLVM dialect MLIR):

```
1 llvm.call @om_gemmini_conv_f32(
2   %out_omtensor, %x_omtensor, %w_omtensor, %stride_i64, %pad_i64
3 ) : (!llvm.ptr, !llvm.ptr, !llvm.ptr, i64, i64) -> ()
```

Understanding the ABI boundary

This is a crucial moment. Before this stage, the compiler used MLIR's internal data structures (memrefs, OMTensorType). After this stage, everything is a raw pointer or integer.

It's like the difference between:

```
1 // Internal compiler representation
2 struct Tensor { float* data; int* shape; int rank; };
3
4 // Stable ABI representation
5 void* tensor; // Opaque pointer
```

The runtime library knows how to interpret the opaque pointer. The compiler-generated code just passes it along.

4.4 Stage 4: LLVM Dialect to LLVM IR

Think of it like disassembling

This stage takes the compiler's internal representation and prints it as standard LLVM assembly. It's like taking the compiler's internal data structures and writing them as text.

4.4.1 What Problem Does This Solve?

MLIR's LLVM dialect is still MLIR syntax. Standard LLVM tools (llc, opt) expect standard LLVM IR syntax. This stage converts from MLIR syntax to LLVM IR syntax.

4.4.2 What It Does

- Reads `resnet18.onnx.mlir` (LLVM dialect)
- Translates to standard LLVM IR text
- Emits `resnet18.ll`

4.4.3 What It Looks Like

```
1 target triple = "riscv64-unknown-linux-gnu"
2
3 declare void @om_gemmini_conv_f32(ptr, ptr, ptr, i64, i64)
4
5 define ptr @run_main_graph(ptr %input) {
6 entry:
7   %0 = call ptr @main_graph_resnet18(ptr %input)
8   ret ptr %0
9 }
```

Listing 2: Stage 4 LLVM IR Output

Reading LLVM IR

This looks like assembly, but with some important differences:

- `define` and `declare` mark function definitions vs. declarations
- `ptr` is a pointer (like `void*`)
- `i64` is a 64-bit integer
- `call` is a function call
- `ret` is a return instruction

The `declare` line means: "This function exists somewhere, but not in this file." The linker will find it in the runtime libraries.

4.5 Stage 5: LLVM IR to RISC-V Object File

Think of it like assembling

This stage is exactly like running an assembler on a `.s` file. LLVM IR becomes machine code for the RISC-V CPU.

4.5.1 What Problem Does This Solve?

LLVM IR is still CPU-independent. We need actual RISC-V machine code that can run on the target hardware.

4.5.2 What It Does

- Instruction selection: chooses RISC-V instructions for each operation
- Register allocation: assigns CPU registers
- Generates RISC-V machine code
- Creates relocatable ELF object file
- Leaves external symbols unresolved (for the linker)
- Emits `resnet18.o`

4.5.3 What It Looks Like

```

1 $ riscv64-linux-gnu-nm -S resnet18.o | grep constant_30_resnet18
2 0000000000000000 00000000000009300 r constant_30_resnet18
3
4 $ riscv64-linux-gnu-nm -u resnet18.o | grep om_gemmini
5      U om_gemmini_conv_f32

```

Listing 3: Checking the Object File

Understanding the Symbol Table

`nm` shows the symbols in the object file:

- `000...` is the address (will be filled in by linker)
- `00000000000009300` is the size (37,632 bytes, the weight size)
- `r` means read-only data (goes to `.rodata`)
- `U` means undefined - this symbol must be provided by another file

The `U` is crucial. It means: "I call this function, but I don't know where it is yet. Linker, please find it for me."

4.6 Stage 6: ONNX-MLIR C Runtime Library

Think of it like compiling `libc`

This stage is like compiling the C standard library for your target architecture. The ONNX-MLIR runtime provides the tensor management functions that the compiled model expects.

4.6.1 What Problem Does This Solve?

The compiled model needs helpers for:

- Creating and destroying tensors
- Accessing tensor data, shape, and strides

- Managing tensor lists (for multiple inputs/outputs)
- Data type conversions

These helpers are provided by the ONNX-MLIR runtime library.

4.6.2 What It Does

- Compiles C source files to RISC-V object files
- Object files: `OMTensor.o`, `OMTensorList.o`, `OnnxDataType.o`, `SmallFPConversion.o`, `OMInstrument.o`
- Archives them into `libcruntime_rv64.a`
- Uses the same ABI flags as the model (`-march=rv64gc -mabi=lp64d`)

4.6.3 What the Runtime Provides

Object File	Key Functions	What It Does
<code>OMTensor.o</code>	<code>omTensorCreateUntyped</code> , <code>omTensorGetDataPtr</code> , <code>omTensorGetShape</code>	Tensor creation, data/shape access, destruction
<code>OMTensorList.o</code>	<code>omTensorListCreate</code> , <code>omTensorListGetSize</code>	List of tensors for inputs/outputs
<code>OnnxDataType.o</code>	<code>OM_DATA_TYPE_SIZE</code>	Data type sizes and names
<code>SmallFPConversion.o</code>	<code>om_f16_to_f32</code> , <code>om_bf16_to_f32</code>	Float16/bfloat16 conversions

4.7 Stage 7: Gemmini Runtime Library

Think of it like a device driver

This is the actual driver for the Gemmini accelerator. It knows how to configure the hardware, move data to scratchpad memory, and execute operations.

4.7.1 What Problem Does This Solve?

The compiled model calls functions like `om_gemmini_conv_f32`. These functions need to be implemented. The Gemmini runtime library implements all accelerator operations.

4.7.2 What It Does

- Compiles `OMRuntimeGemmini.cpp` to `OMRuntimeGemmini.o`
- Includes Gemmini ABI headers for hardware interface
- Archives into `libgemmini_f32_rv64.a`
- Uses C++ compiler with the same ABI flags

4.7.3 What's Inside Each Runtime Function

A typical Gemmini runtime function does:

1. **Extract metadata:** `omTensorGetShape(x)`, `omTensorGetDataPtr(x)`
2. **Validate shapes:** Check dimensions, types, compatibility
3. **Prepare data:** Convert layout (e.g., NCHW to im2col matrix)
4. **Configure accelerator:** Set up scratchpad, data movement
5. **Execute:** Call Gemmini helper functions (`tilled_matmul_auto`)
6. **Write back:** Copy results from scratchpad to output tensor

4.7.4 Implementation Example (Conceptual)

```

1 extern "C" void om_gemmini_conv_f32(
2     OMTensor *out, const OMTensor *x, const OMTensor *w,
3     int64_t stride, int64_t pad) {
4     // Extract metadata
5     float* x_data = (float*)omTensorGetDataPtr(x);
6     float* w_data = (float*)omTensorGetDataPtr(w);
7     float* out_data = (float*)omTensorGetDataPtr(out);
8
9     int64_t* x_shape = omTensorGetShape(x);
10    int64_t* w_shape = omTensorGetShape(w);
11
12    // NCHW: N, C, H, W
13    int64_t N = x_shape[0], C = x_shape[1], H = x_shape[2], W = x_shape[3];
14    int64_t OC = w_shape[0], KH = w_shape[2], KW = w_shape[3];
15
16    // Compute output dimensions
17    int64_t OH = (H + 2*pad - KH) / stride + 1;
18    int64_t OW = (W + 2*pad - KW) / stride + 1;
19
20    // im2col: convert convolution to matrix multiply
21    // ... build im2col matrix of shape (OH*OW) x (C*KH*KW)
22
23    // Use Gemmini helper for the actual compute
24    tiled_matmul_auto(im2col_matrix, weight_matrix, out_matrix,
25                      OH*OW, OC, C*KH*KW, WS);
26
27    // Write output back to OMTensor
28    // ... copy from out_matrix to out_data
29 }

```

Listing 4: Conceptual Runtime Implementation

4.8 Stage 8: Runner Program

4.8.1 What Problem Does This Solve?

We need a `main()` function to test the compiled model. The runner provides a test harness that loads inputs, calls the model, and prints outputs.

4.8.2 What It Does

- Compiles `float_model_runner.c` to `float_model_runner.o`
- Provides `main()` function
- Creates input tensors with dummy data
- Calls `run_main_graph()`
- Prints output scores

4.8.3 What It Looks Like

```

1 int main() {
2     // Create input tensor (1x3x224x224)
3     float* data = malloc(1*3*224*224*sizeof(float));
4     // Fill with dummy data...
5
6     OMTensor* input = omTensorCreateUntyped(
7         data, 4, shape, NULL, ONNX_TYPE_FLOAT, NULL);
8
9     OMTensorList* inputs = omTensorListCreate(&input, 1);
10
11     // Call the compiled model
12     OMTensorList* outputs = run_main_graph(inputs);
13
14     // Get result and print
15     OMTensor* result = omTensorListGetOmtByIndex(outputs, 0);
16     float* scores = (float*)omTensorGetDataPtr(result);
17     printf("Top score: %f\n", scores[0]);
18
19     return 0;
20 }
```

Listing 5: Runner Program Concept

4.9 Stage 9: Static Linking

Think of it like linking a C program

This is exactly like running `gcc -static` on a C program. The linker combines all object files and libraries into a single, self-contained executable.

4.9.1 What Problem Does This Solve?

We have many object files:

- `resnet18.o` - the compiled model
- `float_model_runner.o` - the test harness
- `libgemmini_f32_rv64.a` - Gemmini runtime
- `libcruntime_rv64.a` - ONNX-MLIR runtime

- System libraries (`-lm, libc, libstdc++`)

We need to combine them into one executable that can run on RISC-V.

4.9.2 What It Does

- Reads all object files and archives
- Resolves all symbols (connects calls to definitions)
- Copies code from archives into the executable
- Assigns final addresses to code and data
- Emits the final ELF executable

4.9.3 Link Order Matters!

The linker processes inputs left to right:

```

1 g++ -static -o resnet18_rv64 \
2   float_model_runner.o \      # provides main()
3   resnet18.o \                # provides model, references runtime
4   libgemmini_f32_rv64.a \     # provides om_gemmini_* functions
5   libcruntime_rv64.a \       # provides OMTensor functions
6   -lm                        # math library (sqrtf, expf)

```

Listing 6: Correct Link Order

Why order matters

Think of it like a puzzle:

1. The puzzle (model object) has empty spaces labeled "need function X"
2. The first library (`libgemmini`) fills some spaces
3. The second library (`libcruntime`) fills the remaining spaces
4. If you give the libraries in the wrong order, some spaces remain empty

4.9.4 What the Final Executable Contains

ELF Section	Size (approx)	Contents
<code>.text</code>	1.55 MB	CPU code (model, runtime, system libraries)
<code>.rodata</code>	46.9 MB	Model weights, string constants
<code>.data.rel.ro</code>	0.05 MB	Read-only relocatable data
<code>.data</code>	0.01 MB	Initialized mutable globals
<code>.bss</code>	0.03 MB	Zero-initialized globals

5 Deep Trace: One Convolution and All Runtime Symbols

This section has two levels of tracing. First, it follows one convolution operation through the entire pipeline in detail. That example shows the mechanics: an ONNX operator becomes a runtime function call, the call becomes an unresolved symbol in an object file, and the linker connects it to the Gemmini runtime implementation. After that worked example, a trace matrix lists the same path for all `om_gemmini_*` runtime functions, not only convolution.

Following the Journey

We'll trace a single convolution from ResNet-18's first layer:

- Input shape: 1x3x224x224 (batch, channels, height, width)
- Weight shape: 64x3x7x7 (output channels, input channels, kernel height, width)
- Stride: 2, Padding: 3
- Output shape: 1x64x112x112

This convolution has $64 \cdot 3 \cdot 7 \cdot 7 = 9,408$ weights and does millions of multiply-adds.

5.1 Stage 1: ONNX File to ONNX Dialect

What happens: The frontend parser reads the ONNX model and creates an `onnx.Conv` operation.

What it looks like:

```

1 %y = "onnx.Conv"(%input, %weight, %bias) {
2   dilations = [1, 1],
3   group = 1,
4   kernel_shape = [7, 7],
5   pads = [3, 3, 3, 3],
6   strides = [2, 2]
7 } : (tensor<1x3x224x224xf32>,
8     tensor<64x3x7x7xf32>,
9     tensor<64xf32>) -> tensor<1x64x112x112xf32>

```

What changed: Nothing fundamental. The operation is now in the compiler's internal representation, but it's still a high-level convolution.

Why we need this: The compiler needs to work with the model in a structured format. The ONNX dialect is that format.

5.2 Stage 2: ONNX Dialect to Gemmini/Krnl MLIR

What happens: The compiler checks if this convolution can be accelerated. Since it meets all criteria (float32, group=1, dilation=1, equal stride, symmetric padding), it's replaced with a runtime call.

What it looks like:

```

1 krnl.call @om_gemmini_conv_f32(
2   %output, # memref<1x64x112x112xf32>
3   %input,  # memref<1x3x224x224xf32>
4   %weight, # memref<64x3x7x7xf32>
5   %stride, # 2
6   %padding # 3
7 )

```

What changed: The high-level `onnx.Conv` is gone. It's been replaced with a call to `om_gemmini_conv_f32`.

Why we need this: The compiler has made the decision: "This operation will run on the accelerator." The runtime function knows how to talk to Gemmini.

5.3 Stage 3: Krnl to LLVM Dialect

What happens: The `Krnl` call is converted to an LLVM call. The `memrefs` become `OMTensor` pointers.

What it looks like:

```

1 llvm.call @om_gemmini_conv_f32(
2   %out_omtensor, # !llvm.ptr
3   %x_omtensor,  # !llvm.ptr
4   %w_omtensor,   # !llvm.ptr
5   %stride_i64,   # i64
6   %pad_i64       # i64
7 )

```

What changed: The MLIR-specific types (`memref`, `i64`) are now LLVM types (`ptr`, `i64`). This is the ABI boundary.

Why we need this: The runtime library is written in C/C++, not MLIR. It expects standard C types, not compiler-internal types.

5.4 Stage 4: LLVM Dialect to LLVM IR

What happens: The LLVM dialect is exported as standard LLVM IR.

What it looks like:

```

1 declare void @om_gemmini_conv_f32(ptr, ptr, ptr, i64, i64)
2
3 call void @om_gemmini_conv_f32(
4   ptr %out_omtensor,
5   ptr %x_omtensor,
6   ptr %w_omtensor,
7   i64 2,
8   i64 3
9 )

```

What changed: The syntax is now standard LLVM IR. The `declare` line says "this function exists, but not in this file."

Why we need this: LLVM tools expect standard LLVM IR syntax. This is the last compiler-specific stage before CPU code generation.

5.5 Stage 5: LLVM IR to RISC-V Object

What happens: LLVM generates RISC-V code. The call to `om_gemmini_conv_f32` becomes a relocation.

What it looks like:

```
1 $ riscv64-linux-gnu-nm -u resnet18.o | grep om_gemmini_conv_f32
2          U om_gemmini_conv_f32
```

What changed: The function call is now an unresolved symbol (U) in the object file.

Why we need this: The object file can't know the final address of the runtime function. The linker will fill it in later.

5.6 Stages 6-7: Runtime Libraries

What happens: The runtime libraries are built as separate static archives.

What it looks like:

```
1 $ riscv64-linux-gnu-ar t libgemmini_f32_rv64.a
2 OMRuntimeGemmini.o
3
4 $ riscv64-linux-gnu-objdump -t OMRuntimeGemmini.o | grep
   om_gemmini_conv_f32
5 0000000000000000 g      F .text 00000000 om_gemmini_conv_f32
```

What changed: The runtime function now has an address (tentative) and is marked as global (g).

Why we need this: The linker needs to find the symbol. `OMRuntimeGemmini.o` provides the definition.

5.7 Stage 8: Runner

What happens: The test harness provides `main()` which calls the model.

What it looks like:

```
1 int main() {
2     OMTensorList* inputs = create_input_tensor();
3     OMTensorList* outputs = run_main_graph(inputs);
4     // ...
5 }
```

What changed: We now have a complete program with a starting point.

Why we need this: The final executable needs an entry point. `main()` is that entry point.

5.8 Stage 9: Static Linking

What happens: The linker resolves all symbols. The call to `om_gemmini_conv_f32` is now connected to its implementation.

What it looks like:

```
1 $ riscv64-linux-gnu-nm -S resnet18_rv64 | grep om_gemmini_conv_f32
2 000000000000a1a0 0000000000000b78 T om_gemmini_conv_f32
3
```



```

4 $ riscv64-linux-gnu-objdump -d resnet18_rv64 | grep -A2
   om_gemmini_conv_f32
5 105f8:    jal    23a40 <om_gemmini_conv_f32>

```

What changed:

- The symbol is now defined (T means text section)
- It has a final address: 0x000000000000a1a00
- The call site has the final address: jal 23a40

Why we need this: The executable is now complete. All function calls have addresses. The program can run.

5.9 The Convolution Journey Complete

Stage	Form of the Convolution	Who's Responsible	Key Characteristic
1	ONNX Conv operation	Frontend parser	High-level, still in compiler
2	krnl.call	Gemmini lowering	Replaced with function call
3	"om_gemmini_conv_f32"	Krnl-to-LLVM	ABI boundary crossed
4	llvm.call with OMTensor pointers	LLVM translation	Standard IR, not MLIR
5	LLVM IR call	LLVM translation	U symbol, unresolved
6-7	Relocation in object file	LLVM 11c	Definition exists in archive
8	OMRuntimeGemmini.o	GCC compilation	Entry point provided
9	main() calls run_main_graph	Runner compilation	jal instruction with address
	Final resolved call	Linker	

5.10 Trace Matrix for All Gemmini Runtime Functions

The convolution trace above is the easiest single example, but the same binary logic applies to every runtime entry point. From a CPU programmer's point of view, each `om_gemmini_*` function is just a normal external function: the generated model emits a call, `resnet18.o` contains an unresolved symbol, `OMRuntimeGemmini.o` defines that symbol, and the final static link replaces the unresolved call with a real RISC-V `jal` target.

Not every function below must appear in the ResNet-18 executable. A symbol appears only if the compiled model actually uses the corresponding ONNX operator. The important point is that every function follows the same trace pattern when it is used.

How to trace any runtime function

Replace `$SYM` with any symbol from the table:

```

1 # 1. Did the compiled model request this runtime function?
2 riscv64-linux-gnu-nm -u resnet18.o | grep "$SYM"
3
4 # 2. Does the Gemmini runtime archive provide the implementation?
5 riscv64-linux-gnu-nm -S libgemmini_f32_rv64.a | grep "$SYM"
6
7 # 3. Did the final ELF contain the resolved function?
8 riscv64-linux-gnu-nm -S resnet18_rv64 | grep "$SYM"
9
10 # 4. Where does the final executable call or define it?
11 riscv64-linux-gnu-objdump -d resnet18_rv64 | grep -A4 "$SYM"

```

If step 1 finds a `U` symbol but steps 2 or 3 do not find a definition, the problem is usually missing archive input, wrong archive order, or a mismatch between the generated symbol name and `OMRuntimeGemmini.hpp`.

Table 2: Trace matrix for every public `om_gemmini_*` runtime entry point.

Runtime Symbol	ONNX-style Operation	Execution Path	What to Look For
<code>om_gemmini_matmulinteger_i8i8acc32</code>	MatMulInteger	Quantized integer matrix multiply. This is the most natural Gemmini path because Gemmini is designed around integer matrix operations.	Unresolved model call, definition in <code>OMRuntimeGemmini.o</code> , then Gemmini API calls such as tiled <code>matmul</code> helpers.
<code>om_gemmini_matmulinteger_i8i8acc32_zp</code>	MatMulInteger with zero-points	Same integer matrix multiply path, with extra zero-point tensors used to convert stored quantized values into real signed arithmetic values.	Check that the zero-point tensors are passed as extra <code>OMTensor</code> arguments and that the final symbol name includes <code>_zp</code> .
<code>om_gemmini_qlinearconv_i8</code>	QLinearConv	Quantized convolution. The runtime uses input/weight/output scales and zero-points, then routes the core multiply-accumulate work toward Gemmini.	Trace scale and zero-point <code>OMTensor</code> arguments as well as stride and padding integer arguments.
<code>om_gemmini_qlinearconv_i8_bias</code>	QLinearConv with bias	Quantized convolution plus an i32 bias tensor, usually one bias value per output channel.	Same as <code>QLinearConv</code> , with one additional bias <code>OMTensor</code> in the ABI.
<code>om_gemmini_matmul_f32_hw</code>	MatMul f32 rank-2	Float tensors enter the ABI, then the runtime converts or packs work for the integer-oriented Gemmini hardware path and restores float output semantics.	Look for runtime code that reads <code>OMTensor</code> shapes, prepares matrices, and calls Gemmini <code>matmul</code> helpers.
<code>om_gemmini_matmul_f32_nd_hw</code>	Batched MatMul f32	Higher-rank input is treated as multiple matrix multiplications. The runtime iterates batches and sends each matrix multiply through the Gemmini-backed path.	Check batch-loop logic before the Gemmini <code>matmul</code> helper call.

Runtime Symbol	ONNX-style Operation	Execution Path	What to Look For
om_gemmini_matmul_f16_hw	MatMul f16	Half-precision input is accepted at the ABI boundary. The runtime handles type conversion or packing before using the hardware-oriented matrix path.	Look for f16 conversion and shape handling around the matmul helper.
om_gemmini_conv_f32	Conv f32	The detailed trace above: ONNX Conv becomes a runtime call, then the runtime uses Gemmini-backed convolution or matrix-multiply helpers.	Search each pipeline artifact for <code>om_gemmini_conv_f32</code> .
om_gemmini_conv_f32_bias	Conv f32 with bias	Same Conv path, with a bias tensor added after or during accumulation depending on the runtime helper path.	Trace the extra bias OMTensor argument and verify it reaches the implementation.
om_gemmini_convtranspose_f32	ConvTranspose f32	Transposed convolution, often used for upsampling. The runtime maps the reverse spatial operation into explicit loops or helper routines.	Look for stride, padding, and output-padding scalar arguments.
om_gemmini_convtranspose_f32_bias	ConvTranspose f32 with bias	Transposed convolution with an extra bias tensor.	Same trace as ConvTranspose, with one extra OMTensor argument for bias.
om_gemmini_relu_f32	Relu	CPU scalar fallback path in the Gemmini runtime namespace. It is still linked from the Gemmini runtime archive, but it does ordinary element-wise CPU work.	Expect normal load/compare/store code rather than RoCC command emission.
om_gemmini_batchnorm_f32	BatchNormalization	CPU scalar inference formula using scale, bias, mean, variance, and epsilon.	Trace multiple OMTensor inputs and the encoded epsilon scalar.
om_gemmini_add_f32	Add	CPU scalar or broadcast-compatible element-wise addition.	Check shape/broadcast handling and normal floating-point arithmetic instructions.
om_gemmini_sigmoid_f32	Sigmoid	CPU scalar nonlinear function. It keeps the graph linkable even when no dedicated Gemmini sigmoid instruction exists.	Look for math-library or approximation code, not RoCC instructions.
om_gemmini_mul_f32	Mul	CPU scalar or broadcast-compatible element-wise multiplication.	Trace two input OMTensors and one output OMTensor.
om_gemmini_softmax_f32	Softmax	CPU scalar reduction and normalization over the class dimension.	Look for max/sum/exp-style loops over batch and classes.
om_gemmini_globalavgpool_f32	GlobalAveragePool	CPU scalar pooling over the full spatial area for each channel.	Look for loops over N, C, H, and W and division by spatial element count.
om_gemmini_maxpool_f32	MaxPool	CPU scalar windowed maximum over NCHW input.	Trace kernel, stride, and padding scalar arguments.
om_gemmini_avgpool_f32	AveragePool	CPU scalar windowed average over NCHW input.	Trace kernel, stride, padding, and <code>countIncludePad</code> .

Runtime Symbol	ONNX-style Operation	Execution Path	What to Look For
om_gemmini_gemm_f32	Gemm	General matrix multiply. The runtime handles transpose flags and alpha/beta scalars, then routes the core multiply work through the Gemmini-backed path when supported.	Check transposition flags and encoded alphaBits/betaBits.
om_gemmini_gemm_f32_bias	Gemm with C/bias	Gemm plus a third matrix/tensor input used as C or bias.	Trace the extra C OMTensor and beta handling.
om_gemmini_resize_nearest_f32	Resize nearest	CPU scalar spatial resampling.	Check coordinate and nearest-mode scalar arguments.
om_gemmini_resize_linear_f32	Resize linear	CPU scalar bilinear resampling.	Look for interpolation math and coordinate-mode handling.
om_gemmini_pad_constant_f32	Pad constant	CPU scalar padding with constant zero values.	Trace left/right/top/bottom scalar pad arguments.
om_gemmini_pad_reflect_f32	Pad reflect	CPU scalar padding by reflecting edge-adjacent input values.	Look for index remapping at image boundaries.
om_gemmini_pad_edge_f32	Pad edge	CPU scalar padding by repeating edge values.	Look for clamped index calculations.
om_gemmini_slice_f32	Slice	CPU scalar copy from selected N/C/H/W ranges.	Trace start, end, and step scalar triples for each rank-4 dimension.
om_gemmini_concat_f32	Concat	CPU scalar copy that joins two tensors along one axis.	Trace axis handling and two input OMTensors.
om_gemmini_transpose_f32_hw	Transpose	Runtime reorders rank-4 data according to an explicit permutation. Despite the <code>_hw</code> suffix, inspect the implementation to confirm whether this build uses hardware assistance or CPU copy loops.	Trace <code>perm0</code> through <code>perm3</code> ; confirm the actual implementation path in OMRuntimeGemmini.cpp .
om_gemmini_split_2_f32	Split into 2 outputs	CPU scalar copy from one input tensor into two output tensors along an axis.	Trace multiple output OMTensor pointers before the input pointer.
om_gemmini_split_3_f32	Split into 3 outputs	Same split logic with three output tensors.	Verify the ABI argument order: outputs first, input next, axis last.
om_gemmini_split_4_f32	Split into 4 outputs	Same split logic with four output tensors.	This is useful for ONNX graphs that branch one tensor into several consumers.

How to read the table. The **Execution Path** column tells whether the function is expected to reach Gemmini/RoCC hardware or remain a CPU helper. This distinction matters for performance, but not for linking: both hardware-backed and CPU-fallback

functions are ordinary symbols in `libgemmini_f32_rv64.a`. The linker does not know which functions use hardware; it only knows whether a symbol is defined.

6 Verification and Debugging

6.1 Checking the Final Executable

```

1 # Check file type
2 file resnet18_rv64
3 # Output: ELF 64-bit LSB executable, UCB RISC-V, version 1 (SYSV),
   statically linked
4
5 # Check sections
6 readelf -SW resnet18_rv64 | grep -E '\.text|\.rodata|\.data|\.bss'
7
8 # Check symbols
9 nm -S --size-sort resnet18_rv64 | grep constant_
10
11 # Find runtime calls
12 riscv64-linux-gnu-objdump -d resnet18_rv64 | grep om_gemmini_

```

Listing 7: Basic Verification Commands

6.2 Common Issues and Solutions

Error	Cause and Solution
undefined reference to 'om_gemmini_*'	libgemmini_f32_rv64.a is not linked or is in the wrong order. Add it after <code>resnet18.o</code> .
undefined reference to 'omTensor*'	libcruntime_rv64.a is missing. Link it after libgemmini_f32_rv64.a.
unknown command line argument '-accel'	Use <code>-maccel=Gemmini</code> (note the 'm').
error: cannot find 'onnx-mlir'	Use the Gemmini-enabled build, not the base ONNX-MLIR.
objdump: unrecognized architecture	Use <code>riscv64-linux-gnu-objdump</code> , not host <code>objdump</code> .
.rodata section too large	Add <code>-mcmmodel=medany</code> to LLVM and GCC flags.

6.3 Common Misunderstandings

Frequently Asked Questions

Q: Why can't we just compile to RISC-V directly?

A: The ONNX model isn't code yet; it's a graph description. We need to translate the graph to code.

Q: Where are the weights?

A: They become constant data in the `.rodata` section of the executable. Like global arrays.

Q: Why use two compilers?

A: ONNX-MLIR understands the model structure; GCC understands C/C++ runtime libraries. They meet via the OMTensor ABI.

Q: Why static linking?

A: For embedded/simulator targets, it's easier to have one self-contained executable without shared library dependencies.

Q: Why is the executable so big (50 MB)?

A: Most of it is model weights (46.9 MB) stored as constant data. The code itself is only about 1.5 MB.

Q: How does the accelerator know what to do?

A: The runtime library issues custom RISC-V instructions (RoCC) that the accelerator recognizes. The CPU executes these instructions, and the accelerator handles them.

7 Complete Build Script

Here's the complete script that implements the entire pipeline:

```

1  #!/bin/bash
2  set -e
3
4  ROOT_DIR="/home/lahcen/toolchains/onnx-mlir-xcompile/onnx-mlir"
5  BIN_DIR="$ROOT_DIR/gemmini_toolchain_build/Release/bin"
6  LLVM_BIN="$ROOT_DIR/./llvm-install/bin"
7  OUT_DIR="$ROOT_DIR/gemmini/tests/15-resnet18-float"
8  MODEL_FILE="$ROOT_DIR/gemmini/examples/15-resnet18-float/resnet18.onnx"
9  B=resnet18
10 RISCVC_GCC=riscv64-linux-gnu-gcc
11 RISCVC_GXX=riscv64-linux-gnu-g++
12 FLAGS_ONNX="--maccel=Gemmini --mtriple=riscv64-unknown-linux-gnu"
13 FLAGS_RV="-O2 -march=rv64gc -mabi=lp64d"
14
15 echo "=== Stage 1: ONNX -> ONNX Dialect ==="
16 $BIN_DIR/onnx-mlir $FLAGS_ONNX --EmitONNXIR \
17   -o $OUT_DIR/${B}_onnxir $MODEL_FILE
18
19 echo "=== Stage 2: ONNX -> Krnl/Gemmini ==="
20 $BIN_DIR/onnx-mlir $FLAGS_ONNX --EmitMLIR \
21   -o $OUT_DIR/${B}_krnl $MODEL_FILE
22
23 echo "=== Stage 3: Krnl -> LLVM Dialect ==="
24 $BIN_DIR/onnx-mlir $FLAGS_ONNX --EmitLLVMIR \
25   -o $OUT_DIR/${B} $MODEL_FILE
26
27 echo "=== Stage 4: LLVM Dialect -> LLVM IR ==="
28 $LLVM_BIN/mlir-translate --mlir-to-llvmir \
29   $OUT_DIR/${B}.onnx.mlir -o $OUT_DIR/${B}.ll
30
31 echo "=== Stage 5: LLVM IR -> RISC-V Object ==="
32 $LLVM_BIN/llc $FLAGS_RV --mtriple=riscv64-unknown-linux-gnu \
33   --mcpu=generic-rv64 --mattr=+m,+a,+c,+f,+d \
34   -filetype=obj -relocation-model=static \
35   -o $OUT_DIR/${B}.o $OUT_DIR/${B}.ll
36
37 echo "=== Stage 6: Build ONNX-MLIR C Runtime ==="

```

```

38 mkdir -p $OUT_DIR/rt_obj
39 $RISCV_GCC -c $FLAGS_RV -I $ROOT_DIR/include \
40   $ROOT_DIR/src/Runtime/OnnxDataType.c -o $OUT_DIR/rt_obj/OnnxDataType.o
41 $RISCV_GCC -c $FLAGS_RV -I $ROOT_DIR/include \
42   $ROOT_DIR/src/Support/SmallFPConversion.c -o $OUT_DIR/rt_obj/SmallFPConversion.o
43 $RISCV_GCC -c $FLAGS_RV -I $ROOT_DIR/include \
44   $ROOT_DIR/src/Runtime/OMTensor.c -o $OUT_DIR/rt_obj/OMTensor.o
45 $RISCV_GCC -c $FLAGS_RV -I $ROOT_DIR/include \
46   $ROOT_DIR/src/Runtime/OMTensorList.c -o $OUT_DIR/rt_obj/OMTensorList.o
47 $RISCV_GCC -c $FLAGS_RV -I $ROOT_DIR/include \
48   $ROOT_DIR/src/Runtime/OMInstrument.c -o $OUT_DIR/rt_obj/OMInstrument.o
49
50 riscv64-linux-gnu-ar rcs $OUT_DIR/libcruntime_rv64.a \
51   $OUT_DIR/rt_obj/OnnxDataType.o \
52   $OUT_DIR/rt_obj/SmallFPConversion.o \
53   $OUT_DIR/rt_obj/OMTensor.o \
54   $OUT_DIR/rt_obj/OMTensorList.o \
55   $OUT_DIR/rt_obj/OMInstrument.o
56
57 echo "=== Stage 7: Build Gemmini Runtime ==="
58 mkdir -p $OUT_DIR/gem_rt_obj
59 GEMMINI_HW_ABI_DIR=$ROOT_DIR/src/Accelerators/Gemmini/Runtime/gemmini-hardware-abi
60 GEMMINI_HW_ABI_INCLUDE=$GEMMINI_HW_ABI_DIR/include
61 $RISCV_GXX -c $FLAGS_RV -I $ROOT_DIR/include -I $ROOT_DIR \
62   -I $GEMMINI_HW_ABI_DIR -I $GEMMINI_HW_ABI_INCLUDE \
63   $ROOT_DIR/src/Accelerators/Gemmini/Runtime/OMRuntimeGemmini.cpp \
64   -o $OUT_DIR/gem_rt_obj/OMRuntimeGemmini.o
65 riscv64-linux-gnu-ar rcs $OUT_DIR/libgemmini_f32_rv64.a \
66   $OUT_DIR/gem_rt_obj/OMRuntimeGemmini.o
67
68 echo "=== Stage 8: Compile Runner ==="
69 $RISCV_GCC -c $FLAGS_RV -I $ROOT_DIR/include \
70   $ROOT_DIR/gemmini/examples/tools/float_model_runner.c \
71   -o $OUT_DIR/float_model_runner.o
72
73 echo "=== Stage 9: Static Link ==="
74 $RISCV_GXX $FLAGS_RV -static \
75   -o $OUT_DIR/${B}_rv64 \
76   $OUT_DIR/float_model_runner.o \
77   $OUT_DIR/${B}.o \
78   $OUT_DIR/libgemmini_f32_rv64.a \
79   $OUT_DIR/libcruntime_rv64.a \
80   -lm
81
82 echo "=== Verification ==="
83 file $OUT_DIR/${B}_rv64
84 readelf -SW $OUT_DIR/${B}_rv64 | grep -E '\.text|\.rodata|\.data|\.bss'
85 nm -S $OUT_DIR/${B}_rv64 | grep -c 'constant.*_resnet18'
86
87 echo "Done: $OUT_DIR/${B}_rv64"

```

Listing 8: Full Pipeline Script

8 Quick Reference: What Each Command Does

Command	CPU Programmer's Understanding	What It Actually Does
<code>onnx-mlir</code>	Compiler frontend - read the model	Parses ONNX binary to MLIR ONNX dialect
<code>-EmitONNXIR</code>		
<code>onnx-mlir -EmitMLIR</code>	Compiler optimization - inline/runtime calls	Replaces ops with runtime calls (KrnI)
<code>onnx-mlir -EmitLLVMIR</code>	Compiler backend - generate assembly	Lowers to LLVM dialect
<code>mlir-translate</code>	Compiler output - write assembly text	Exports LLVM dialect to LLVM IR
<code>-mlir-to-llvmir</code>		
<code>llc -filetype=obj</code>	Assembler	Generates RISC-V machine code
<code>riscv64-linux-gnu-gcc -c</code>	Compile C source	Compiles runtime C files to objects
<code>riscv64-linux-gnu-ar rcs</code>	Archiver	Creates static library from objects
<code>riscv64-linux-gnu-g++ -static</code>	Linker	Combines objects/libraries into executable

9 Summary: What We've Built

- **Input:** A ResNet-18 ONNX model (46 MB, neural network description)
- **Process:** Nine compilation and linking stages
- **Output:** `resnet18_rv64` (50 MB static ELF executable)
- **Contents:** Model code (1.55 MB), Weights (46.9 MB), Runtime libraries (0.2 MB), System libraries
- **What it does:** Runs ResNet-18 inference using the Gemmini accelerator
- **How:** Compiler-generated code calls Gemmini runtime functions; runtime issues RoCC instructions to the accelerator

Final Thought

This whole pipeline is like:

1. You write a high-level algorithm (the neural network)
2. The compiler translates it to assembly
3. The runtime library provides the hardware-specific details
4. The linker puts it all together
5. The result is a program that runs on the target hardware

The only difference is that "target hardware" includes both a RISC-V CPU and a Gemmini accelerator. The compiler handles the CPU code generation; the runtime

library handles the accelerator code generation. The pipeline is the bridge between the model and the hardware.